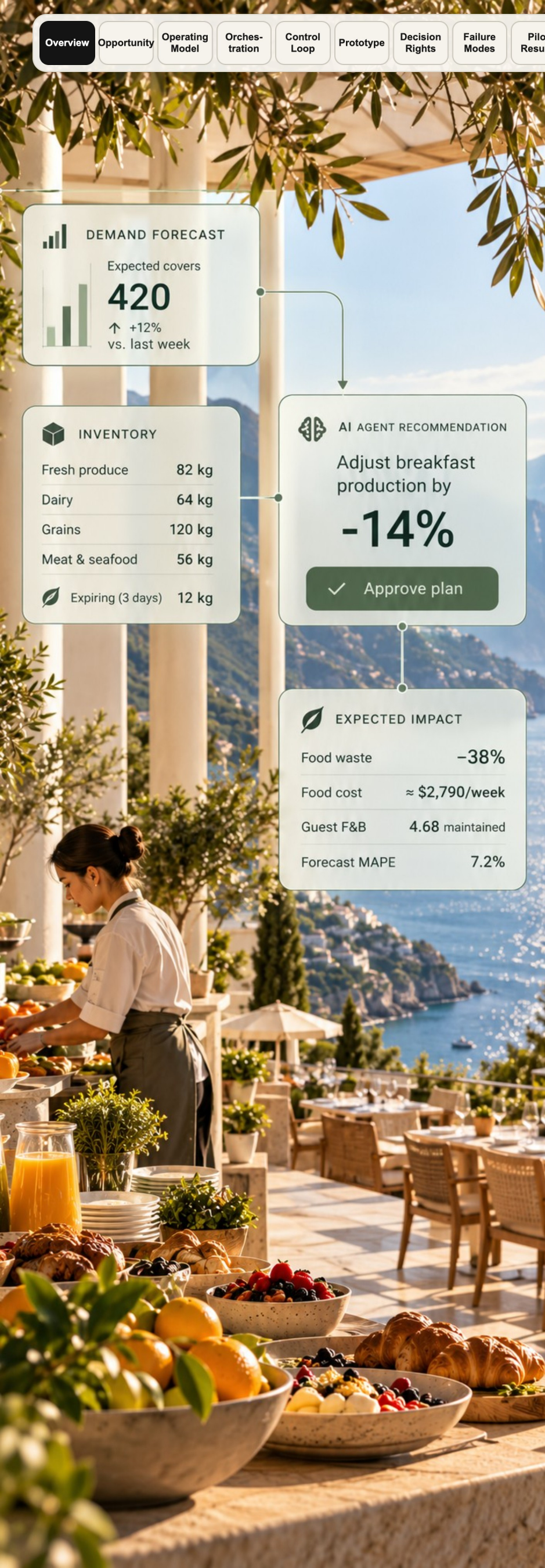




DEMAND FORECAST

Expected covers

420

↑ +12% vs. last week

INVENTORY

Fresh produce	82 kg
Dairy	64 kg
Grains	120 kg
Meat & seafood	56 kg
Expiring (3 days)	12 kg

AI AGENT RECOMMENDATION

Adjust breakfast production by

-14%

✓ Approve plan

EXPECTED IMPACT

Food waste	-38%
Food cost	≈ \$2,790/week
Guest F&B	4.68 maintained
Forecast MAPE	7.2%

HOSPITALITY × SUSTAINABILITY × AGENTIC AI

CAN AI MAKE HOSPITALITY WASTE LESS?

Designing an agentic operating system to reduce hotel food waste without compromising profitability or guest experience.



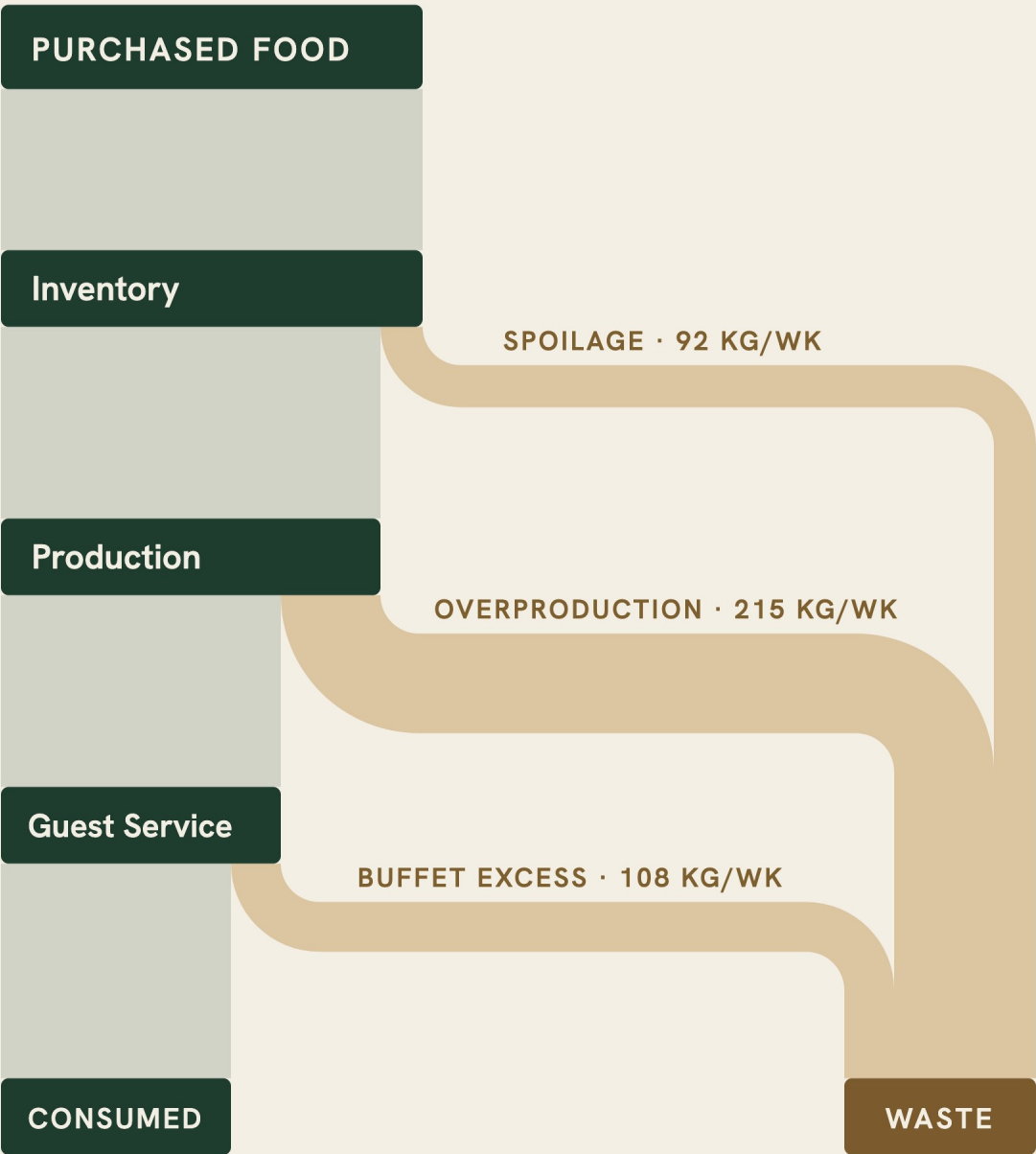
A case study by Amelia Wolfire

Developed in the MIT executive program *Implementing Agentic AI: Building Your Organizational Playbook*

MIT Sloan School of Management × MIT Schwarzman College of Computing
• 2026

Food waste is more than a sustainability problem.

Every wasted ingredient represents purchasing, forecasting, production, labor, and operational capacity that **failed to create guest value**.



DECISIONS MADE EVERY DAY

- 01 How many guests are coming?
- 02 What will they consume?
- 03 What inventory already exists?
- 04 How much should the kitchen prepare?
- 05 What was wasted yesterday?
- 06 What should change tomorrow?

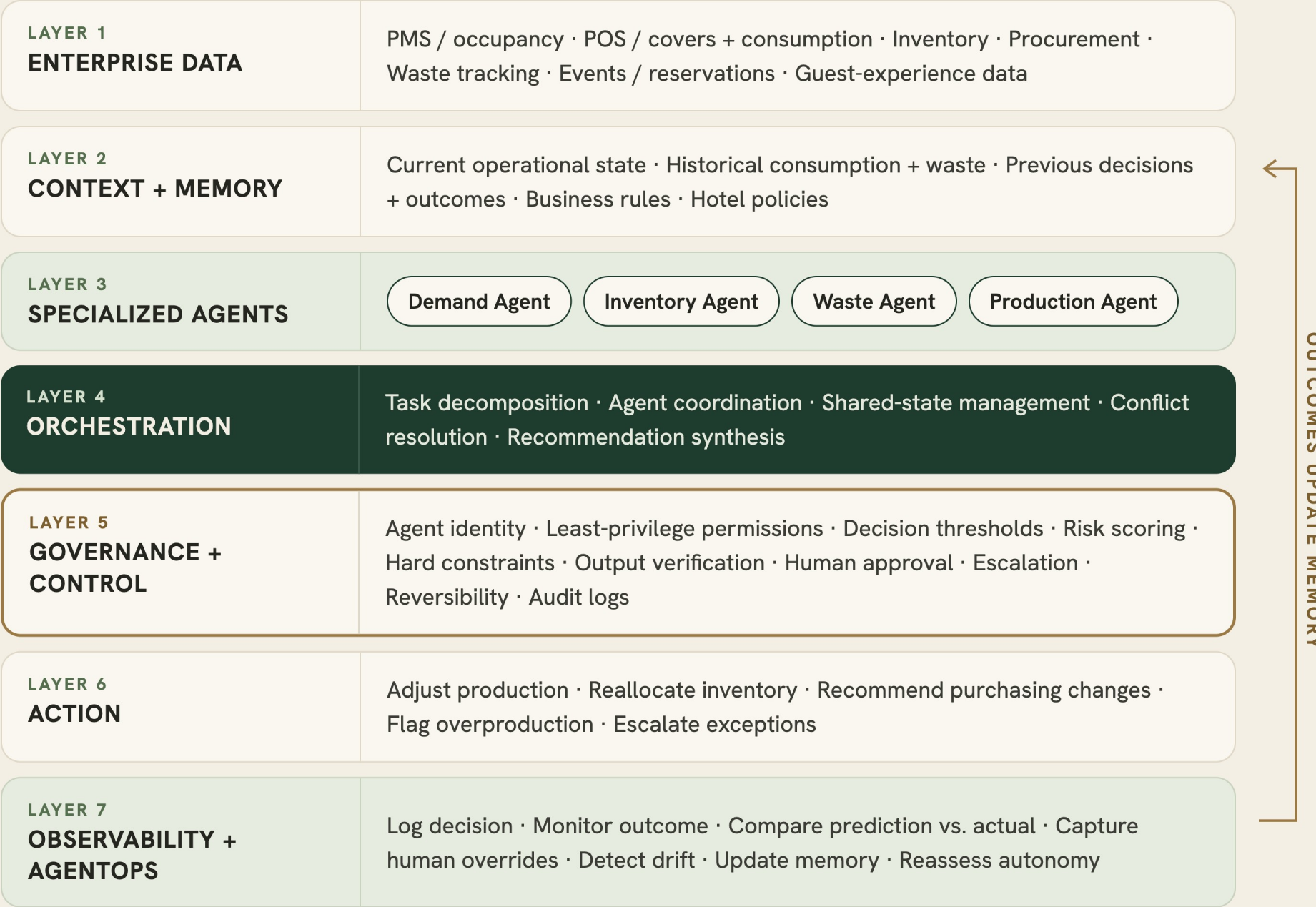
The problem isn't a lack of data.
It's disconnected decision-making.

02 / THE OPERATING MODEL

A Governed, Closed-Loop Agentic Architecture

OBJECTIVE
Minimize food waste + unnecessary operating cost

SUBJECT TO
Guest-experience floor · Food-safety constraints · Service-level requirements · Inventory availability · Purchasing policy · Human decision rights



AI is the mechanism,
not the goal.

I framed the work as a constrained business optimization problem, not an AI project.

03 / ORCHESTRATION

Why Four Specialists Beat One General Model.

I split the decision into four specialist roles, each with only the access it needs, and designed one orchestrator to combine them.

DEMAND AGENT

“What demand should we expect?”

READ PMS · reservations · POS

WRITE none · cannot purchase inventory

INVENTORY AGENT

“What do we already have?”

READ inventory · procurement

WRITE inventory recommendation

WASTE AGENT

“Where and why are we losing food?”

READ waste logs · POS consumption

WRITE waste insights

PRODUCTION AGENT

“What should we prepare?”

READ approved plan only

WRITE production system, within threshold

ORCHESTRATOR

COORDINATE agent outputs · **EXECUTE** no unrestricted purchasing authority

“How do these signals combine into one operational decision?”

Shared context

Dependencies

Conflicting recommendations

Business constraints

Risk

Approval requirements

Conflict example · Demand expects +12% covers, but Waste flags chronic pastry overproduction. The orchestrator raises hot-line prep, holds pastry volume, and routes the change for chef approval.

Every agent has only the access its role requires.

A Closed-Loop Decision System, Not a One-Time Prediction.

EVERY SERVICE RUNS THE LOOP

- 01

PERCEIVE
Pull occupancy, reservations, POS, inventory and waste into shared context.
- 02

REASON
Specialized agents interpret demand, stock and waste patterns.
- 03

PLAN
The orchestrator combines their signals into one production plan.
- 04

GOVERN
Risk, reversibility, permissions and policy decide who may act.
- 05

ACT
Execute within bounds, or route to a human for approval.
- 06

OBSERVE
Capture what was actually served, consumed and wasted.
- 07

EVALUATE
Compare prediction with outcome; record overrides and errors.
- 08

ADAPT
Update memory, recalibrate thresholds, reassess autonomy.

↻ Step 08 feeds step 01 of the next service

SIGNAL → CONTEXT → AGENT ANALYSIS → PLAN → RISK CHECK → APPROVAL → ACTION → OUTCOME → MEMORY

Every material agent decision should be traceable.

DECISION TRACED-0418 · Sat breakfast

Agents		Demand · Inventory · Waste → Orchestrator
Inputs		231 of 250 rooms · 420 expected covers
Recommendation		Breakfast production –14%
Risk classification		Medium · exceeds ±10% range
Approval status		Manager approved
Outcome		409 actual covers · 2.7% error
08:02	PMS occupancy retrieved	PERCEIVE
08:02	POS history retrieved	PERCEIVE
08:03	Inventory state verified	PERCEIVE
08:03	Demand forecast generated	REASON
08:03	Waste pattern identified	REASON
08:04	Production plan proposed	PLAN
08:04	Risk threshold evaluated	GOVERN
08:04	Human approval requested	GOVERN
08:07	Manager approved	ACT
08:08	Production plan updated	ACT
14:30	Actual outcome captured	OBSERVE
14:31	Outcome written to memory	ADAPT

From Strategy to System.

I translated the operating model into orchestration logic: shared context, parallel agents, policy and risk checks, human approval, execution and logging.

orchestration.py

```
1 import asyncio
2
3 async def run_breakfast_cycle(hotel_id: str):
4     # 1 · Shared context from enterprise systems
5     context = await ContextManager.load(hotel_id,
6         sources=["PMS", "POS", "inventory", "waste"])
7     if context.completeness < 0.95:
8         return await Escalation.abstain(context)
9
10    # 2 · Specialist agents run in parallel
11    demand, inventory, waste = await asyncio.gather(
12        DemandAgent.run(context),
13        InventoryAgent.run(context),
14        WasteAgent.run(context))
15
16    # 3 · Orchestrator resolves one plan
17    plan = await Orchestrator.coordinate(
18        demand=demand, inventory=inventory, waste=waste,
19        constraints=PolicyEngine.rules)
20
21    # 4 · Governance decides who may act
22    risk = RiskEngine.score(plan)
23    decision = Governance.evaluate(
24        plan=plan, risk=risk,
25        permissions=AgentIdentity.scope("production"))
26
27    # 5 · Act within decision rights
28    if decision.requires_human:
29        await ApprovalQueue.route(plan, to="F&B manager")
30    else:
31        await ProductionAgent.execute(plan)
32
33    # 6 · Every material decision is traceable
34    AgentOps.log(context, plan, risk, decision)
```

\$ run_breakfast_cycle("H-250")

CONTEXT	8/8 sources · 100%
AGENTS	3 parallel calls
PLAN	breakfast -14%
RISK	MEDIUM
POLICY	0 violations
DECISION	requires_human
ROUTED	F&B manager
APPROVED	08:07 → executed
LOGGED	D-0418

Why a human? A -14% change exceeds the ±10% range the agent may execute alone.

- WHAT EACH BLOCK SHOWS
- 1

Shared context + completeness check
- 2

Asynchronous specialist agents
- 3

Orchestration under policy rules
- 4

Risk scoring + least-privilege permissions
- 5

Human approval or bounded execution
- 6

AgentOps logging for every decision

Conceptual implementation pattern — prototype, not production deployment. A production build would connect these agents to live PMS/POS, inventory, procurement and waste-tracking systems through governed APIs.

Autonomy Is Assigned by Decision, Not by Agent.

DECISION	AUTHORITY
LOW RISK · REVERSIBLE	
Flag overproduction	Agent executes
Recommend inventory transfer	Agent executes within threshold
Small prep adjustment ($\leq \pm 10\%$)	Agent executes within range
MEDIUM RISK	
Larger production adjustment	Agent recommends · manager approves
Purchasing adjustment	Agent recommends · manager approves
HIGH RISK · LOW REVERSIBILITY	
Major purchase cancellation	Human decision
Supplier change	Human decision
Food-safety exception	Mandatory escalation
Major banquet production change	Human approval

REVERSIBILITY

HIGHLY REVERSIBLE

Tomorrow's prep recommendation

MODERATELY REVERSIBLE

Inventory transfer

LOW REVERSIBILITY

Supplier purchase commitment

IRREVERSIBLE · HIGH CONSEQUENCE

Disposal or safety-related action

CONFIDENCE × RISK

LOW RISK

HIGH RISK

HIGH CONF.

LOW CONF.

HIGH CONFIDENCE · LOW RISK

DELEGATE

HIGH CONFIDENCE · HIGH RISK

HUMAN APPROVAL

LOW CONFIDENCE · LOW RISK

RECOMMEND

LOW CONFIDENCE · HIGH RISK

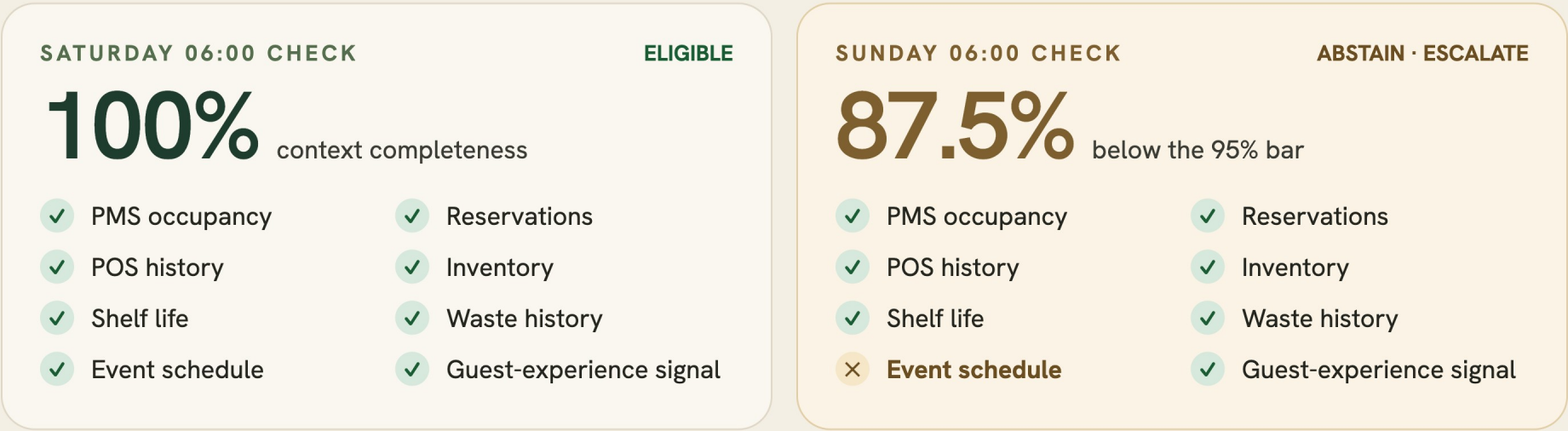
ESCALATE / ABSTAIN

CONFIDENCE ≠ AUTHORITY

High confidence never grants permission to act. Authority depends on risk, reversibility, permissions and policy.

RISK + REVERSIBILITY + CONFIDENCE + POLICY → DECISION AUTHORITY

A Mature Agent Knows When Not to Act.



Context completeness score = required sources present ÷ required sources. Modeled threshold: **≥ 95% for delegated execution**. Missing critical context means the agent abstains and escalates.

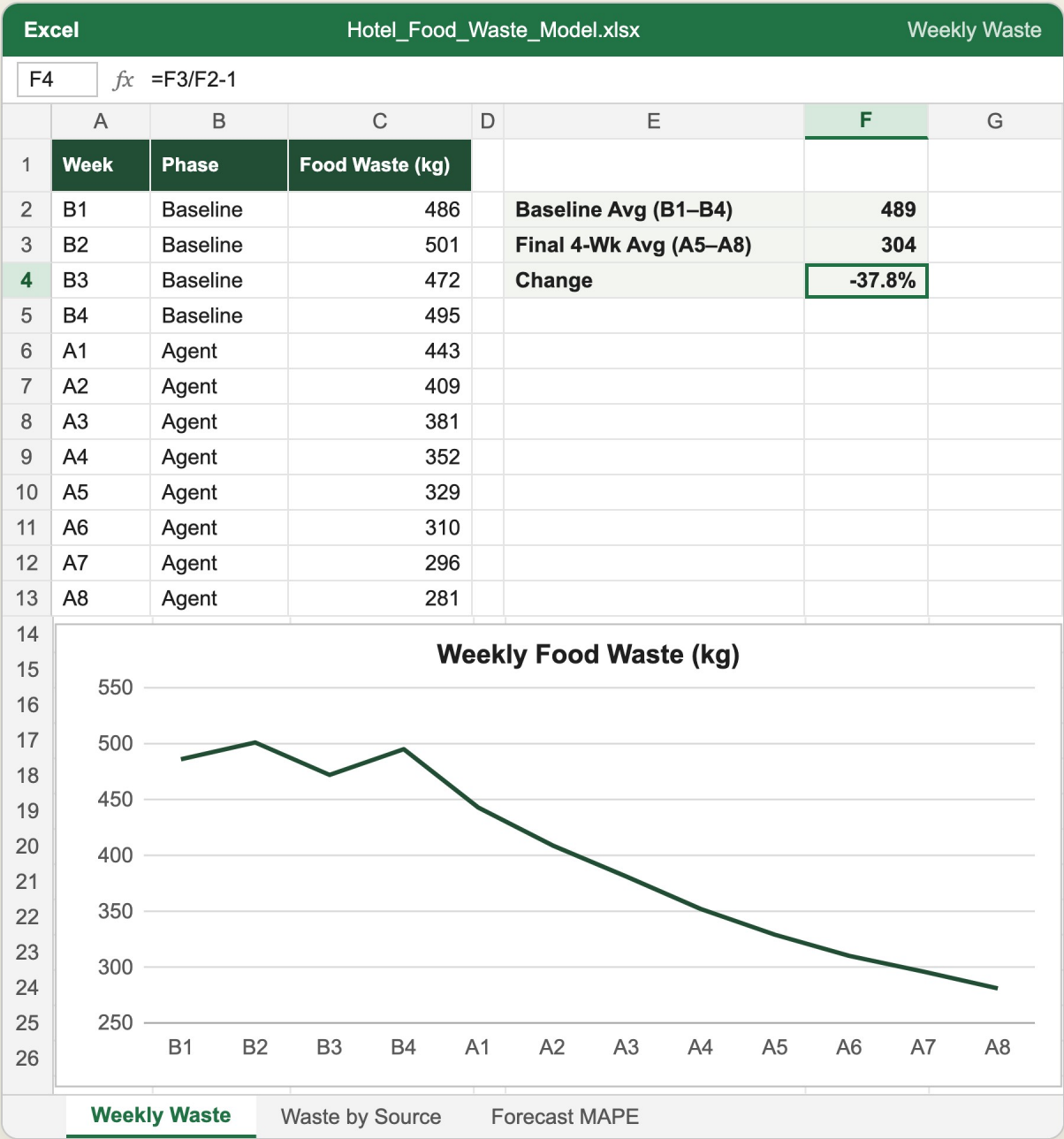
FAILURE	DETECTION	RESPONSE
Missing PMS data	Context check	Abstain
POS API failure	Tool monitoring	Fallback + escalate
Inventory feed stale	Timestamp check	Block execution
Abnormal event demand	Distribution shift	Human review
Low confidence	Confidence threshold	Recommend only
Conflicting agents	Orchestrator conflict	Escalate
High-risk action	Risk classification	Human approval
Policy violation	Hard constraint	Block
Guest score deterioration	Experience guardrail	Reduce autonomy
Rising overrides	AgentOps drift signal	Reduce autonomy + investigate

Knowing when to abstain is a capability, not a failure.

Food Waste Falls as Decisions Improve.

12-WEEK MODELED PILOT 250 rooms 4 baseline + 8 agent weeks 17 core metrics

Applied case study · MIT Sloan × MIT Schwarzman College of Computing, *Implementing Agentic AI* (2026)



489 KG

Baseline weekly average

↓

304 KG

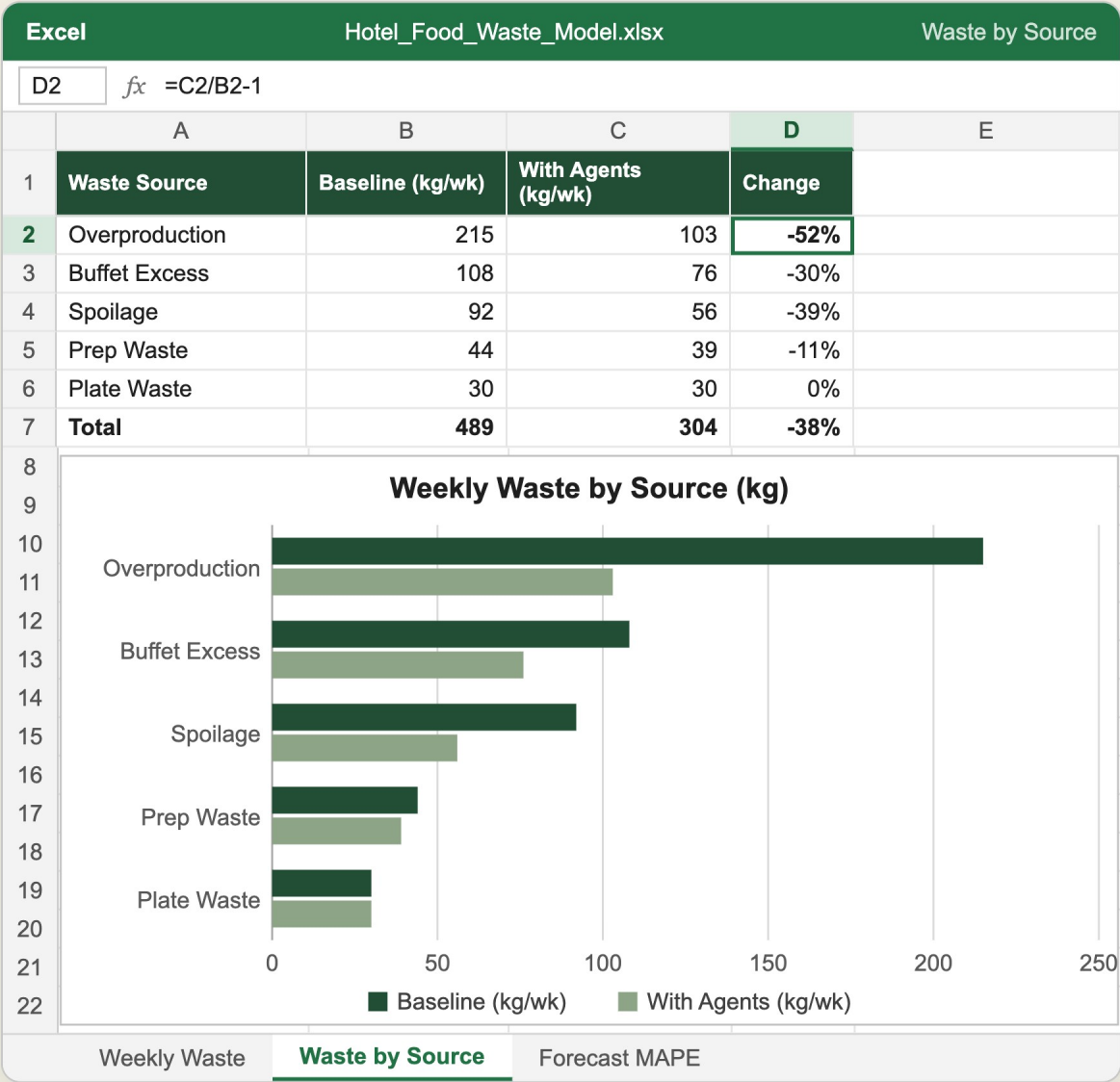
Final 4-week average

-38%

MODELED FOOD WASTE

I modeled how connecting demand, production, inventory and waste data progressively removes unnecessary food production.

The Biggest Opportunity Happens Before Food Reaches the Guest.



-52%

Overproduction

-39%

Spoilage

-30%

Buffet Excess

Prep Waste -11%

Plate Waste — unchanged

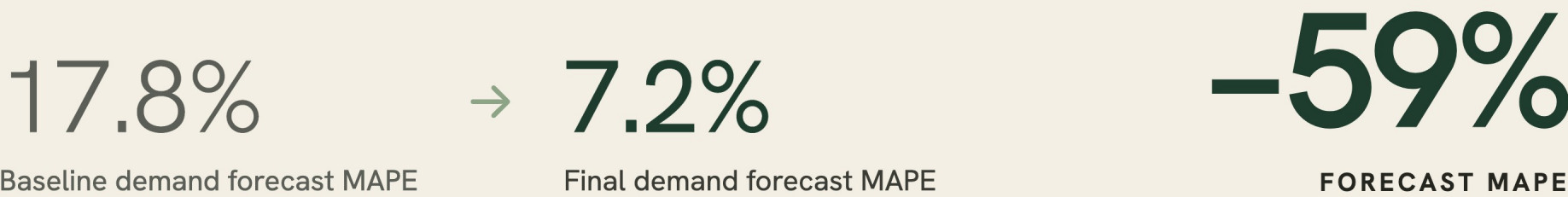
WHAT THE DATA SUGGESTS

The greatest modeled gains occur in waste categories influenced by forecasting, purchasing, inventory, and production decisions.

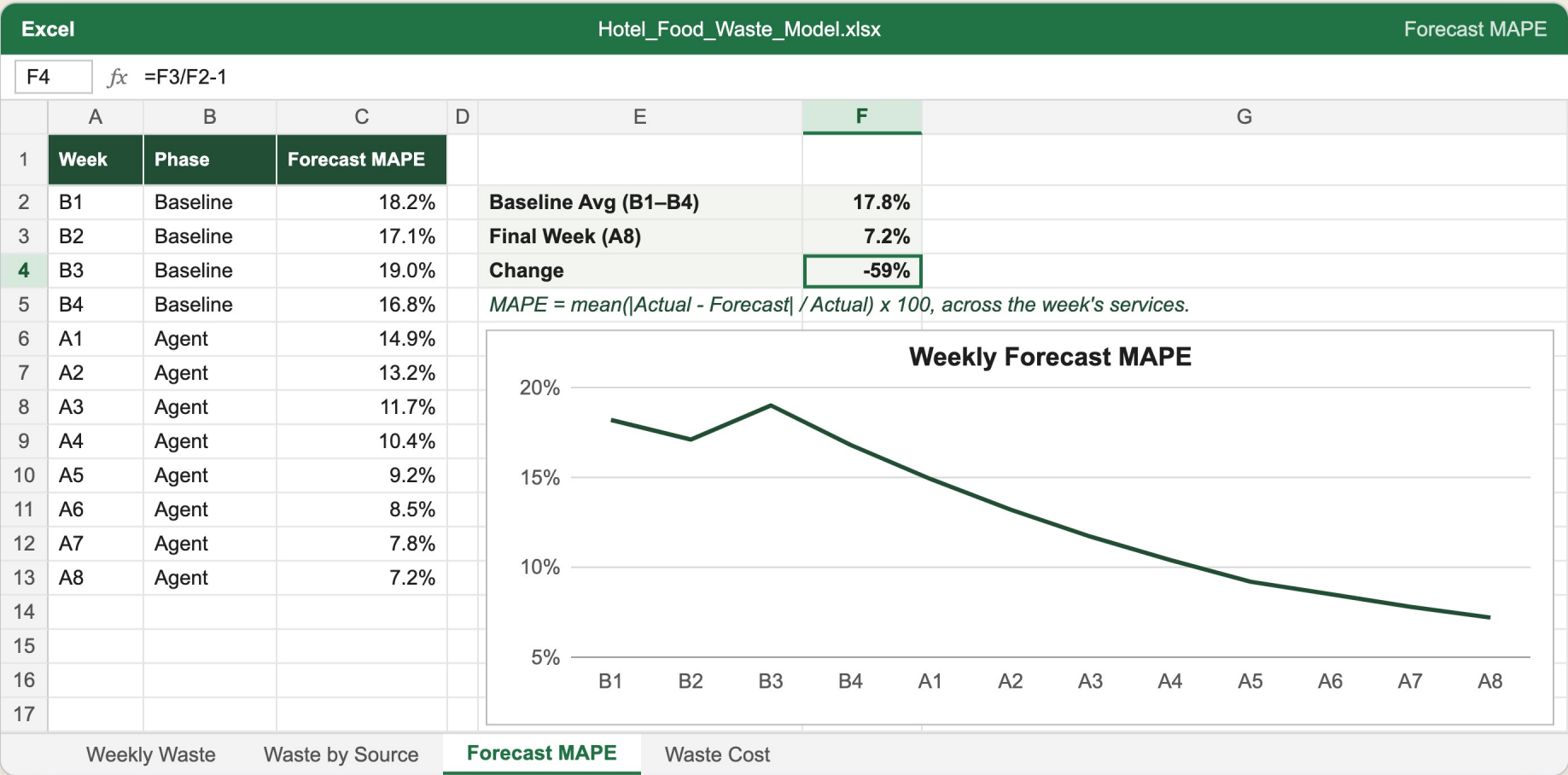
Plate waste remains largely unchanged because the system is designed primarily around operationally controllable waste.

Solve the decision upstream.
Prevent the waste downstream.

Better Forecasts Create Better Production Decisions.



MAPE · Mean Absolute Percentage Error = $\text{mean}(| \text{Actual} - \text{Forecast} | / \text{Actual}) \times 100$



The agent does not reduce waste directly.
It improves the decisions that create waste.

Reduce Excess. Not Experience.

-43%

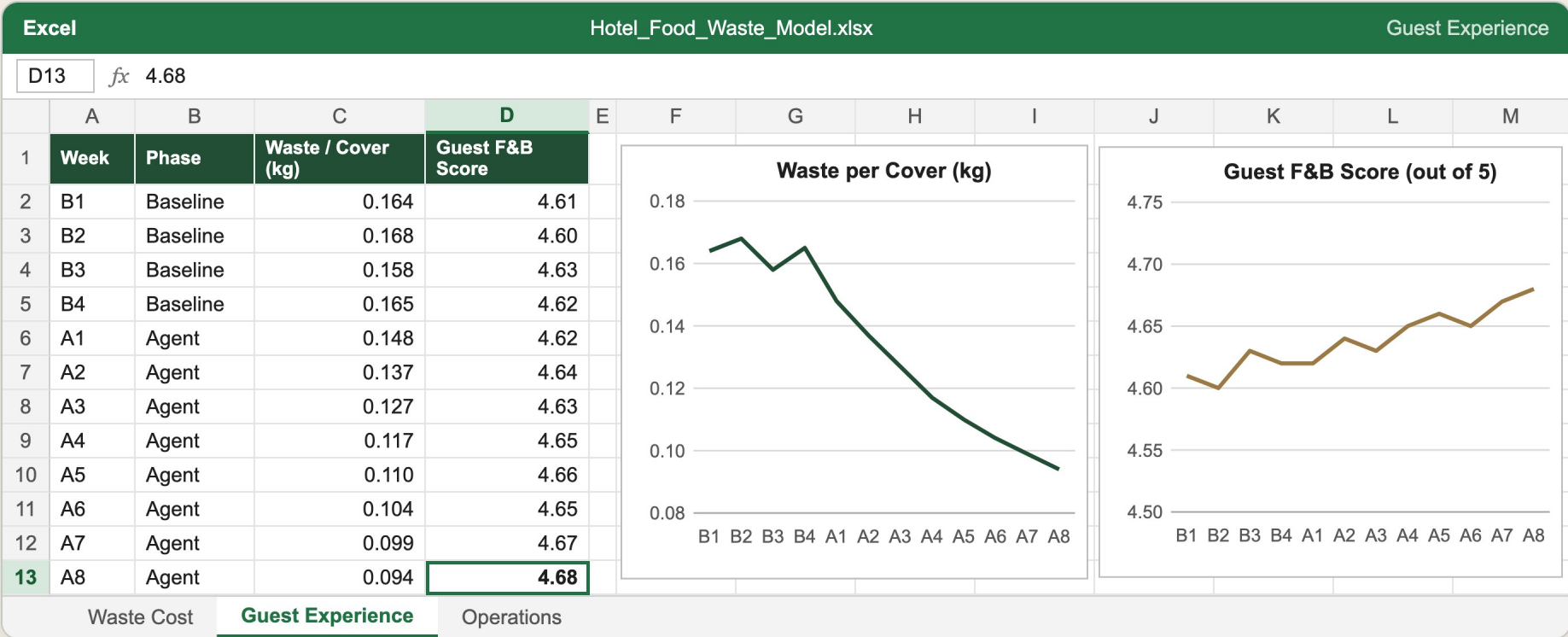
WASTE / COVER

kg per guest served, baseline avg → week A8

4.61 → 4.68

GUEST F&B SATISFACTION

Score out of 5, week B1 → week A8



WASTE ↓ | EXPERIENCE PROTECTED

Efficiency only creates hospitality value if the guest experience is protected.

Is the Agent Useful, Safe, Reliable, Valuable — and Ready for More?

QUALITY

Recommendation acceptance

88.3%

≥ 80%

Forecast MAPE

7.2%

≤ 10%

Recommendation outcome quality*

91%

≥ 85%

SAFETY

Escalation recall

95.5%

≥ 90%

Escalation precision

91.3%

≥ 85%

Policy violations*

0

= 0

Correct abstention rate*

92%

≥ 90%

RELIABILITY

Tool / API reliability

99.4%

≥ 99%

Context completeness*

97.8%

≥ 95%

Human override rate

9.0%

≤ 15%

Execution failure rate*

0.6%

≤ 1%

BUSINESS

Food waste reduction

-37.8%

Waste cost avoided

\$145K

per year

Manager hours saved

8.2

per week

Guest F&B score

4.68

≥ 4.60

Week A8 values. * Modeled illustrative metric; all others from the pilot AgentOps and model sheets.

ESCALATION RECALL

74% → 95%

Of all situations that should have reached a human, how many did the system identify?

Low recall = dangerous events may be missed.

ESCALATION PRECISION

74% → 91%

Of everything the agent escalated, how many actually warranted intervention?

Low precision = humans receive unnecessary escalations.

Good governance requires both.

WHY HUMANS OVERRODE

166 overrides, A1-A8

Missing context

38

Special event

31

Forecast disagreement

29

Guest-experience concern

24

Inventory constraint

19

Policy requirement

14

Agent error

11

OVERRIDE → DIAGNOSIS → SYSTEM IMPROVEMENT

e.g. "Missing context" → add the banquet and group-block feed to the context layer.

DRIFT WATCH · POST-PILOT SCENARIO

9%

10%

11%

23%

1.5x trailing avg

DRIFT SIGNAL DETECTED

→ Check data quality

→ Examine context changes

→ Review agent decisions + tool failures

→ Recalibrate thresholds

→ Temporarily reduce autonomy

Human intervention becomes learning data.

Autonomy Is Earned Through Evidence.

I built a gate in Excel that tests every threshold at once, so no single strong metric can grant more authority.

Excel Hotel_Food_Waste_Model.xlsx Readiness Gate													
K17 fx =IF(AND(B17>=\$C\$2,C17>=\$C\$3,D17>=\$C\$4,E17>=\$C\$5,F17<=\$C\$6,G17<=\$C\$7,H17>=\$C\$8,I17=\$C\$9),"PASS","HOLD")													
	A	B	C	D	E	F	G	H	I	J	K	L	M
11	Week	Acceptance	Recall	Precision	Tool Reliability	Override Rate	MAPE	Guest Score	Policy Violations	Failed Checks	Gate	First Blocker	Autonomy Score
12	A1	61.0%	74.1%	74.1%	96.7%	29.0%	14.9%	4.62	0	6	HOLD	Acceptance	73.2
13	A2	66.7%	77.8%	77.8%	97.4%	25.0%	13.2%	4.64	0	6	HOLD	Acceptance	77.1
14	A3	72.2%	82.1%	82.1%	98.1%	20.9%	11.7%	4.63	0	6	HOLD	Acceptance	81.2
15	A4	76.0%	85.2%	85.2%	98.5%	18.2%	10.4%	4.65	0	5	HOLD	Acceptance	84.1
16	A5	79.7%	88.5%	85.2%	98.7%	14.8%	9.2%	4.66	0	3	HOLD	Acceptance	86.4
17	A6	83.3%	92.0%	88.5%	98.9%	12.9%	8.5%	4.65	0	1	HOLD	Tool reliability	89.2
18	A7	86.3%	92.6%	92.6%	99.1%	10.8%	7.8%	4.67	0	0	PASS	—	91.4
19	A8	88.3%	95.5%	91.3%	99.4%	9.0%	7.2%	4.68	0	0	PASS	—	92.6
AgentOps Readiness Gate Control Room Overrides													

WEEK A6 · HOLD

98.9% vs 99%

Every other threshold passed. **One reliability miss was enough to hold authority** — the gate first passes in A7.

WEEK A8 CHECK · ILLUSTRATIVE THRESHOLDS · ALL MUST HOLD

Acceptance ≥ 80% ✓

Esc. recall ≥ 90% ✓

Esc. precision ≥ 85% ✓

Tool reliability ≥ 99% ✓

Override rate ≤ 15% ✓

Forecast MAPE ≤ 10% ✓

Guest F&B ≥ 4.60 ✓

Policy violations = 0 ✓

RESULT: PASS

A1-A6 returned HOLD; A7-A8 PASS.

AUTONOMY SCORE CONSTRUCTION



Acceptance · Escalation recall · Escalation precision · Tool reliability · Inverse override rate

Case-study composite index; illustrative weighting. It informs the review but never grants authority by itself.

AUTHORITY LADDER



AUTONOMOUS

Not the goal. The end state is Delegated, with human approval gates for high-risk decisions.

The Business Case Includes the AI's Own Cost.

CONSERVATIVE

\$26K

net / year · 23.5-month payback

BASE CASE

\$75K

net / year · 8-month payback

PILOT RESULT

\$136K

net / year · 4.4-month payback

ANNUAL P&L · PILOT RESULT	
VALUE	
Food cost avoided	\$145,000
Coordination capacity redeployed	\$19,188
Gross benefit	\$164,188
AGENTIC AI OPERATING COST	
Model / API usage	\$6,000
Monitoring + observability	\$6,000
Human review of approvals	\$4,680
Maintenance + evaluation	\$12,000
Annual AI cost	(\$28,680)
Net annual benefit	\$135,508
I modeled one-time costs of \$40K integration + \$10K training. Year-1 ROI 109% · value = 5.7× AI run cost. All cost inputs are illustrative.	

Excel		Hotel_Food_Waste_Model.xlsx		Sensitivity
C12	fx	=C10-C11		
	A	B	C	D
1	Scenario Driver	Conservative	Base	Upside
2	Waste reduction	15%	25%	38%
3	Adoption rate	80%	90%	100%
4	AI operating-cost multiplier	125%	100%	100%
5	Food purchasing volume index	100%	100%	100%
6				
7	RESULTS			
8	Food cost avoided	\$46,080	\$86,400	\$145,000
9	Coordination capacity	\$15,350	\$17,269	\$19,188
10	Gross benefit	\$61,430	\$103,669	\$164,188
11	AI operating cost	\$35,850	\$28,680	\$28,680
12	Net annual benefit	\$25,580	\$74,989	\$135,508
13	Payback (months)	23.5	8.0	4.4
14				
Overrides		Economics	Sensitivity	

GROSS BENEFIT - AI OPERATING COST = NET BENEFIT → ROI → PAYBACK

WHAT MOVES THE ANSWER · ONE-WAY SENSITIVITY AROUND THE \$75K BASE



Value is most sensitive to waste-reduction performance, not to what the AI costs to run.

An Operating-Model Change, Not a Software Install.

I assigned a named human owner to every component that shapes a decision.

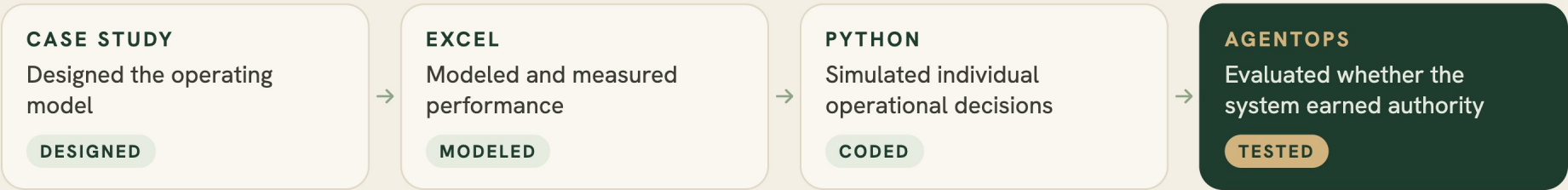
WHO OWNS WHAT F&B Leadership BUSINESS OUTCOME OWNER Waste, food cost and guest-experience targets. Operations WORKFLOW OWNER How recommendations enter kitchen routines. IT / Data DATA + INTEGRATION OWNER PMS, POS, inventory and procurement feeds. AI / Product AGENT PERFORMANCE OWNER Models, prompts, evaluation and AgentOps. Risk / Governance CONTROL OWNER Thresholds, permissions, policy and audit. Human Manager EXCEPTION DECISION-MAKER Approves, overrides and escalates in the moment.	AI BILL OF MATERIALS What must be inventoried and governed in production <table><tr><th>COMPONENT</th><th>OWNER</th></tr><tr><td>Foundation model</td><td>AI / PRODUCT</td></tr><tr><td>Agent framework</td><td>AI / PRODUCT</td></tr><tr><td>Prompts + instructions</td><td>AI / PRODUCT</td></tr><tr><td>Tools</td><td>IT / DATA</td></tr><tr><td>APIs</td><td>IT / DATA</td></tr><tr><td>Data sources</td><td>IT / DATA</td></tr><tr><td>Memory stores</td><td>IT / DATA</td></tr><tr><td>Policies</td><td>RISK / GOV.</td></tr><tr><td>Agent identities</td><td>RISK / GOV.</td></tr><tr><td>Permissions</td><td>RISK / GOV.</td></tr><tr><td>Evaluation metrics</td><td>AI / PRODUCT</td></tr><tr><td>Human owners</td><td>F&B LEADERSHIP</td></tr></table>	COMPONENT	OWNER	Foundation model	AI / PRODUCT	Agent framework	AI / PRODUCT	Prompts + instructions	AI / PRODUCT	Tools	IT / DATA	APIs	IT / DATA	Data sources	IT / DATA	Memory stores	IT / DATA	Policies	RISK / GOV.	Agent identities	RISK / GOV.	Permissions	RISK / GOV.	Evaluation metrics	AI / PRODUCT	Human owners	F&B LEADERSHIP
COMPONENT	OWNER																										
Foundation model	AI / PRODUCT																										
Agent framework	AI / PRODUCT																										
Prompts + instructions	AI / PRODUCT																										
Tools	IT / DATA																										
APIs	IT / DATA																										
Data sources	IT / DATA																										
Memory stores	IT / DATA																										
Policies	RISK / GOV.																										
Agent identities	RISK / GOV.																										
Permissions	RISK / GOV.																										
Evaluation metrics	AI / PRODUCT																										
Human owners	F&B LEADERSHIP																										

If no one owns a component,
no one governs the decisions it shapes.

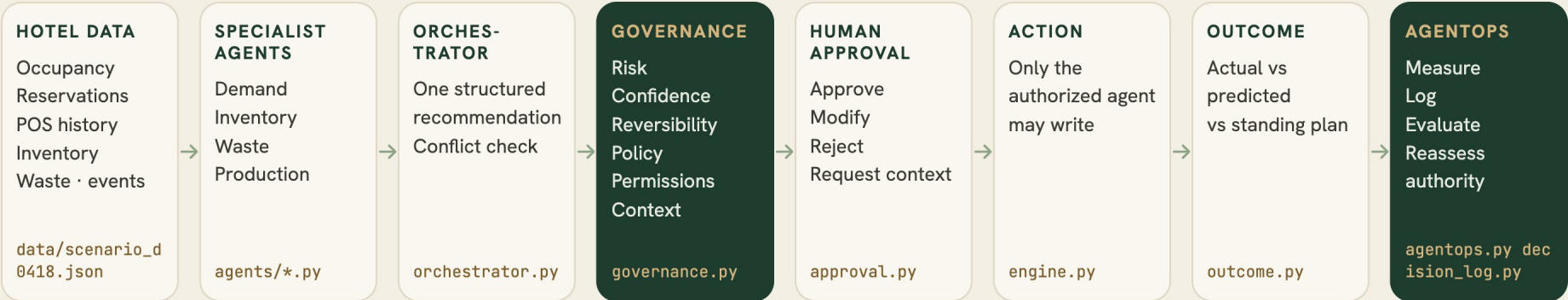
16 / IMPLEMENTATION SIMULATION

From Concept to Execution.

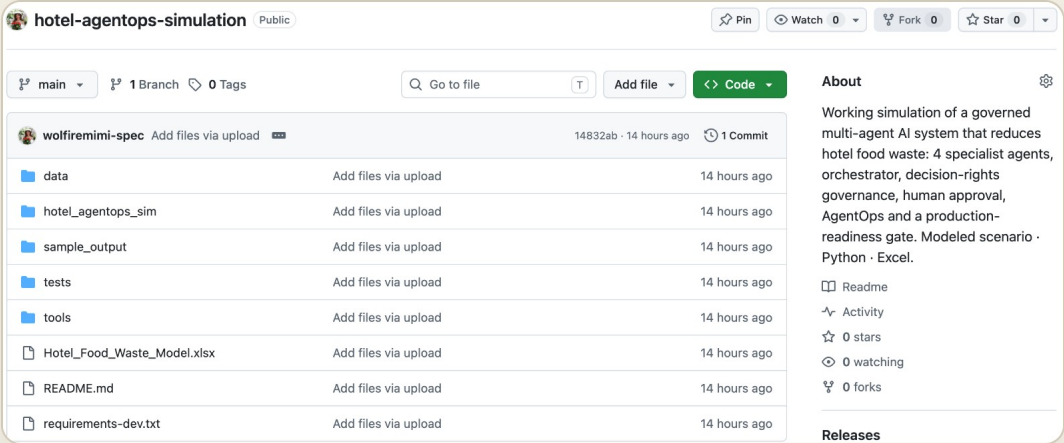
I translated the conceptual operating model into an executable Python simulation of the hotel decision loop.



THE DECISION LOOP I CODED · EACH STEP IS A FILE IN THE REPOSITORY



REAL SCREENSHOT · PUBLIC GITHUB REPOSITORY



WHAT I BUILT

- ✓ Four specialist agents · least-privilege access
- ✓ Orchestration · one recommendation + conflict check
- ✓ Governance engine · decides who may act
- ✓ Human approval · approve · modify · reject · context
- ✓ Decision logging · 24 columns + full trace
- ✓ Outcome simulation · actual vs predicted
- ✓ AgentOps evaluation · metrics with sample size
- ✓ Readiness logic · 8 thresholds from Excel
- ✓ Failure-mode testing · 3 tests + 10-row matrix
- ✓ Override learning · evidence, never auto-retrain

What remains conceptual: live PMS / POS / inventory connections, trained forecasting models (the agents are transparent rule-based Python) and production deployment. Scenario data is modeled.

The architecture described the system.
The simulation tested how it behaves.

One Decision. End to End.

Decision **D-0418** · Saturday breakfast · every value below is output from the executable simulation.

01 / PERCEIVE Hotel data	231 of 250 rooms · 1.819 covers per room (last 4 Saturdays) · 8 of 8 sources, context 100% · 6 kg yogurt near expiry · brunch guarantee 140 → 95			
02 / SPECIALISTS Four agents	DEMAND 420 covers, +12% vs 375 · confidence 92.7%	INVENTORY Carry-over: 4 kg fruit, 2 kg cold cuts	WASTE Pastry leftover >20% in 4 of 4 Saturdays · hot line out of stock 2 of 4	PRODUCTION Draft plan 188 → 165 kg
03 / ORCHESTRATE Recommendation	Reduce breakfast production 12.2% (188 → 165 kg): hot line +14% · pastry -34% · fruit -20% · cold cuts -16%. Conflict (demand +12%, plan -12%) resolved by waste evidence · confidence 92.7%			
04 / GOVERN Authority	Risk MEDIUM (beyond the ±10% delegated range) · reversibility HIGH · policy PASS → Agent recommends · manager approves CONFIDENCE ≠ AUTHORITY Governance log: "High confidence (93%) does not grant authority for MEDIUM risk"			
05 / HUMAN Decision	APPROVED at 08:07 (scripted demo approver; in run mode a person decides)			
06 / ACT Action	Production Agent writes the approved plan → EXECUTED (it cannot write without authorization)			
07 / OBSERVE Outcome	409 actual vs 420 predicted covers · 2.7% error · waste 23.3 kg vs 47.3 kg on the standing plan · no stockouts			
08 / EVALUATE AgentOps	Decision log + trace file · APE 0.0269 · escalation correct · gate (thresholds read from the Excel Readiness Gate, week A8) PASS → DELEGATED · n = 1 service			

REAL SCREENSHOT · `python -m hotel_agentops_sim trace D-0418` · RUN IN GITHUB CODESPACES

```
@wolfiremimi-spec → /workspaces/hotel-agentops-simulation (main) $ python -m hotel_agentops_sim trace D-0418
TRACE D-0418 (INPUT → AGENT → GOVERNANCE → HUMAN → ACTION → OUTCOME → KPI)

INPUT      completeness 100.0% · {'pms_occupancy': 'ok', 'reservations': 'ok', 'pos_history': 'ok', 'inventory': 'ok', 'shelf_life': 'ok', 'waste_histo
ry': 'ok', 'event_schedule': 'ok', 'guest_experience_signal': 'ok'}
AGENT      Reduce breakfast production by 12.2% (188 → 165 kg): Hot line +14%; Pastry & bread -34%; Fruit & yogurt -20%; Cold cuts & cheese -16%
. Hot line stocked out recently, so it is raised.
why: Expected covers up 12% vs last Saturday (375 → 420). Historical waste shows chronic overproduction in Pastry & bread (avg leftover 30%)
confidence 92.7% · risk MEDIUM · reversibility HIGH
GOVERNANCE Context completeness 100.0% (threshold 95%)
Policy: all hard constraints pass
Risk MEDIUM · reversibility HIGH · confidence 92.7%
Decision rights for MEDIUM risk → Agent recommends · manager approves
High confidence (93%) does not grant authority for MEDIUM risk
Agent conflict: Demand +12% but production draft -12%. Resolved: Waste Agent shows chronic overproduction in Pastry & bread, so the plan rai
ses what guests eat and cuts what they leave.
Current autonomy level: DELEGATED
FINAL: Agent recommends · manager approves
HUMAN      APPROVE by F&B Manager (demo) at 08:07
ACTION     Reduce breakfast production by 12.2% (188 → 165 kg): Hot line +14%; Pastry & bread -34%; Fruit & yogurt -20%; Cold cuts & cheese -16% → EXEC
UTED
OUTCOME    409 actual covers vs 420 predicted (2.7% error) · waste 23.3 kg vs 47.3 kg on the standing plan
KPI        {'forecast_ape': 0.0269, 'escalated': True, 'escalation_required': True}
@wolfiremimi-spec → /workspaces/hotel-agentops-simulation (main) $ █
```

The concept pages illustrated -14%; the executable simulation computed -12.2% from its modeled inputs.

Every material AI decision
can be traced from signal to outcome.

I Tested When the AI Should Not Act.

SCENARIO 01

Missing critical context

87.5% vs ≥ 95%

PMS occupancy removed · 7 of 8 sources

CONTEXT CHECK FAILS

↓

ABSTAIN

↓

ESCALATE

↓

LOG EVENT

0 executions attempted. No recommendation is issued; the kitchen keeps its standing plan.

SCENARIO 02

Reliability below threshold

98.9% vs ≥ 99%

Tool reliability · week A6 row from the Excel workbook

7 OF 8 THRESHOLDS PASS

↓

HOLD · FIRST BLOCKER: TOOL RELIABILITY

↓

AUTONOMY: SUPERVISED

No additional authority. The two low-risk actions that ran delegated now need manager approval. Same all-must-pass logic as the Excel Readiness Gate.

SCENARIO 03

Guest-experience guardrail

4.55 vs ≥ 4.60

Guest F&B score · test input

GUARDRAIL FAILS · HOLD

↓

FLAG + ESCALATE

↓

REDUCE AUTONOMY: BOUNDED

↓

HUMAN REVIEW

Efficiency stops outranking the guest. Low-risk actions drop to recommend-only; human-approved decisions still proceed.

GOVERNANCE TESTING

20

automated tests · OK (1 skipped: needs openpyxl)

3 / 3

failure tests passed

10 / 10

failure modes from page 8 executed, not described

REAL SCREENSHOT · python -m unittest -v · RUN IN GITHUB CODESPACES

```
●@wolfiremimi-spec → /workspaces/hotel-agentops-simulation (main) $ python -m unittest -v
test_d0418_reproduces_case_study_trace (tests.test_simulation.EndToEnd.test_d0418_reproduces_case_study_trace) ... ok
test_every_record_has_a_trace (tests.test_simulation.EndToEnd.test_every_record_has_a_trace) ... ok
test_human_modification_is_policy_checked (tests.test_simulation.EndToEnd.test_human_modification_is_policy_checked) ... ok
test_missing_context_refuses_to_act (tests.test_simulation.EndToEnd.test_missing_context_refuses_to_act) ... ok
test_outcome_depends_on_the_human_decision (tests.test_simulation.EndToEnd.test_outcome_depends_on_the_human_decision) ... ok
test_confidence_is_not_authority (tests.test_simulation.Governance.test_confidence_is_not_authority) ... ok
test_low_confidence_recommends_only (tests.test_simulation.Governance.test_low_confidence_recommends_only) ... ok
test_low_risk_is_delegated_only_when_gate_passes (tests.test_simulation.Governance.test_low_risk_is_delegated_only_when_gate_passes) ... ok
test_missing_context_abstains (tests.test_simulation.Governance.test_missing_context_abstains) ... ok
test_no_agent_may_write_purchasing (tests.test_simulation.Governance.test_no_agent_may_write_purchasing) ... ok
test_stale_inventory_blocks_execution (tests.test_simulation.Governance.test_stale_inventory_blocks_execution) ... ok
test_unresolved_conflict_escalates (tests.test_simulation.Governance.test_unresolved_conflict_escalates) ... ok
test_agent_cannot_read_outside_its_permissions (tests.test_simulation.LeastPrivilege.test_agent_cannot_read_outside_its_permissions) ... ok
test_production_agent_cannot_write_without_authorization (tests.test_simulation.LeastPrivilege.test_production_agent_cannot_write_without_authorization) ... ok
test_a6_holds_on_tool_reliability_alone (tests.test_simulation.ReadinessGate.test_a6_holds_on_tool_reliability_alone) ... ok
test_guest_breach_reduces_autonomy (tests.test_simulation.ReadinessGate.test_guest_breach_reduces_autonomy) ... ok
test_matches_workbook_for_every_pilot_week (tests.test_simulation.ReadinessGate.test_matches_workbook_for_every_pilot_week) ... ok
test_no_averaging (tests.test_simulation.ReadinessGate.test_no_averaging) ... ok
test_override_drift_reduces_autonomy (tests.test_simulation.ReadinessGate.test_override_drift_reduces_autonomy) ... ok
test_parameters_match_workbook (tests.test_simulation.SourceOfTruth.test_parameters_match_workbook) ... skipped 'openpyxl not installed'

Ran 20 tests in 0.021s

OK (skipped=1)
●@wolfiremimi-spec → /workspaces/hotel-agentops-simulation (main) $
```

A trustworthy AI system isn't defined only by what it can do.
It's also defined by knowing when not to act.

Abstention is a capability, not a failure.

WHAT I DESIGNED

USE CASE STRATEGY → MULTI-AGENT ARCHITECTURE → ORCHESTRATION → GOVERNANCE → PYTHON PROTOTYPE → EXCEL EVALUATION → AGENTOPS → PYTHON SIMULATION → **SCALE STRATEGY**

I IDENTIFIED THE PROBLEM	Hotel food waste and fragmented operational decision-making.
I DESIGNED THE SYSTEM	A multi-agent architecture coordinating demand, inventory, production and waste.
I PROTOTYPED THE LOGIC	Python-based agent orchestration and governance.
I BUILT THE IMPLEMENTATION SIMULATION	I translated the conceptual orchestration model into an executable Python simulation testing agent decisions, governance, human approval, failure modes and AgentOps.
I BUILT THE MEASUREMENT MODEL	An Excel pilot analysis connecting agent behavior to operational KPIs.
I DESIGNED THE GOVERNANCE	Decision rights, thresholds, escalation, permissions, reversibility and human oversight.
I TESTED THE BUSINESS CASE	Sustainability, profitability, operational efficiency and guest experience, net of the AI's own cost.
I DEFINED HOW IT SCALES	AgentOps, production-readiness gates, monitoring and evidence-based autonomy.

I didn't design a chatbot.
I designed an operating system for decisions.

The goal isn't maximum autonomy. It's better decisions, measurable business value, and the right level of autonomy for the risk.

Amelia Wolfire

Hospitality ▪ Sustainability ▪ Brand & Business Strategy ▪ Agentic AI

MIT Sloan × MIT Schwarzman College of Computing · Implementing Agentic AI Executive Program, 2026

Modeled 12-week pilot scenario | Case-study projections, not realized hotel performance.